

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 18

INDEXING:

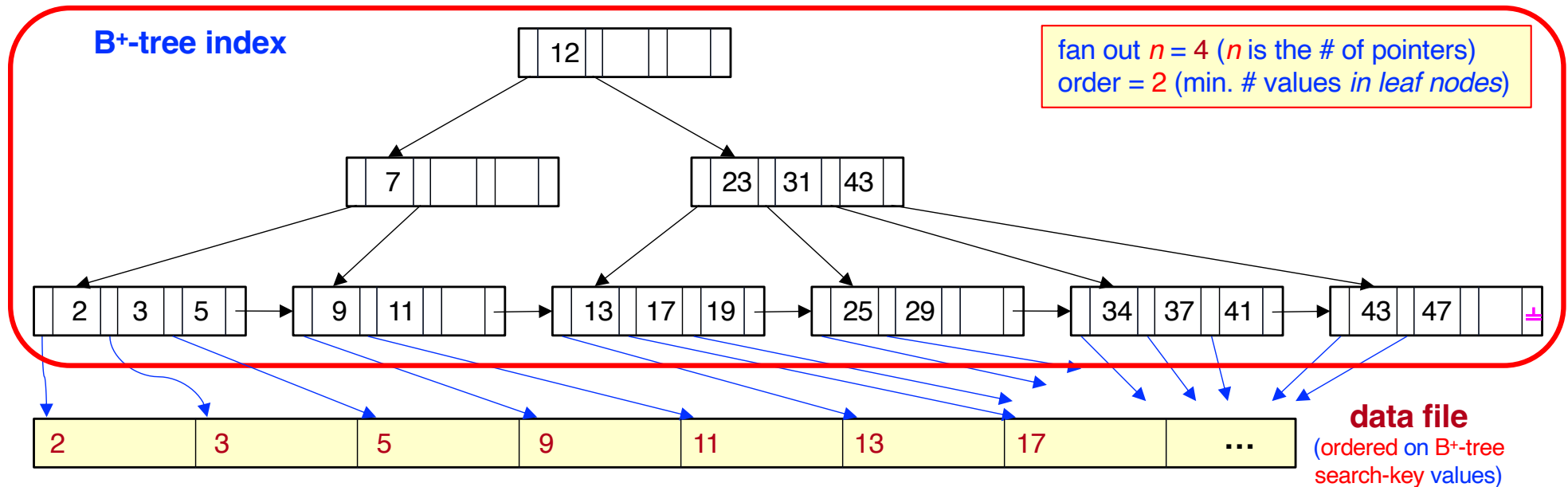
B+-TREE INDEX AND HASH INDEX

B⁺-TREE INDEX

- **Disadvantage of ordered-index (*aka* indexed-sequential) files**
 - Performance degrades as the file grows due to overflow pages.
 - Periodic reorganization of the entire file is required.
- **Advantage of B⁺-tree index files**
 - Balanced tree $\Rightarrow$ every path from root to leaves is the same length.
 - Automatically reorganizes itself with only small, local changes in the presence of insertions and deletions.
 - Reorganization of entire file is not required to maintain performance.
- **Disadvantage of B⁺-trees**
 - Extra insertion, deletion and space overhead.

**The advantages of B⁺-trees far outweigh their disadvantages.
They are used extensively in all commercial DBMSs.**

B⁺-TREE: STRUCTURE AND PROPERTIES



Each non-leaf (internal) node has between

- $\lceil n/2 \rceil$ and n pointers ($\lceil n/2 \rceil - 1$ (min) and $n - 1$ (max) values).
- In the example: between 2 and 4 pointers (1 and 3 values).

Special cases for root (top-most node):
if non-leaf - min 1 value.
if leaf - min 0 values.

Each leaf node is at least half full, i.e., it has between

- $\lceil (n-1)/2 \rceil + 1$ and n pointers ($\lceil (n-1)/2 \rceil$ (min) and $n - 1$ (max) values).

In the example: between 3 and 4 pointers (2 and 3 values).

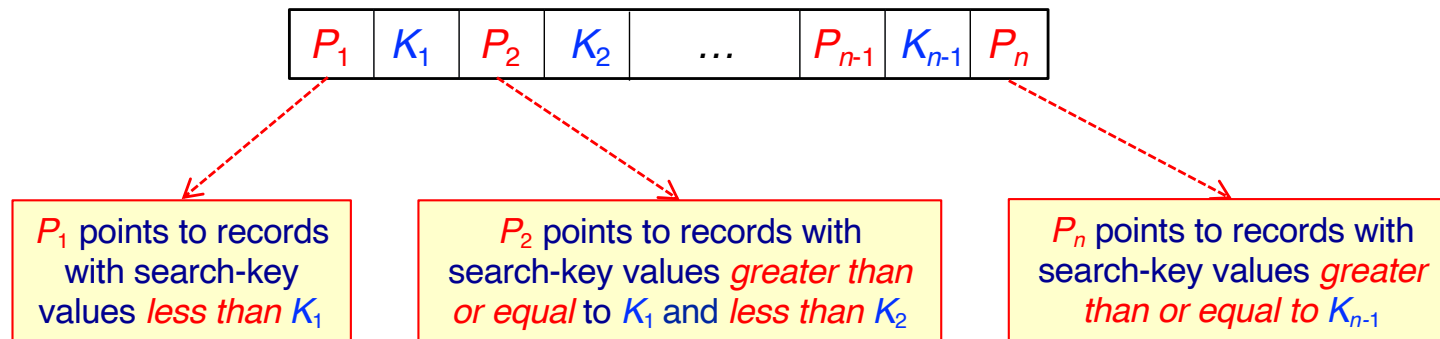
☞ Each node has one more pointer than values.

B⁺-TREE: STRUCTURE AND PROPERTIES (CONT'D)

- **Balanced tree**
 - All **paths** from the root node to the leaf nodes are the **same length**.
- **Fan-out**
 - The **maximum number of pointers/children** in each node, denoted ***n***.
- **B⁺-tree order**
 - The value $\lceil (n-1)/2 \rceil$ is called the **order** and corresponds to the **minimum number of values in a leaf node**.
- **Special cases**
 - If the **root is not a leaf**, it has **at least 2 children** (i.e., it has **at least 1 value**).
 - If the **root is a leaf** (i.e., there are no other nodes in the tree), it can have **between 0 and (*n*−1) values**.

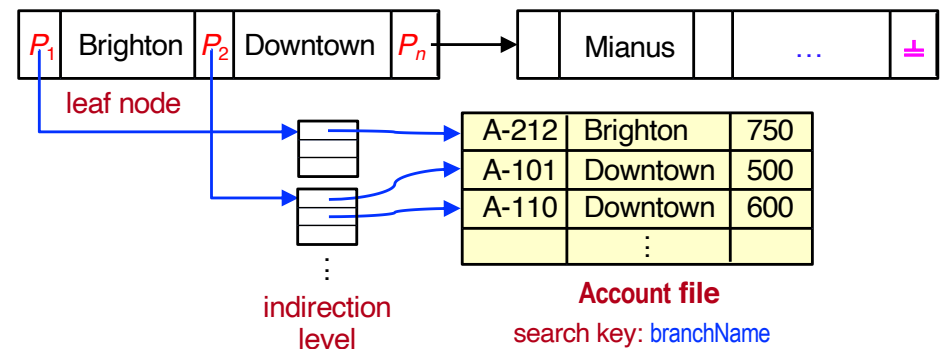
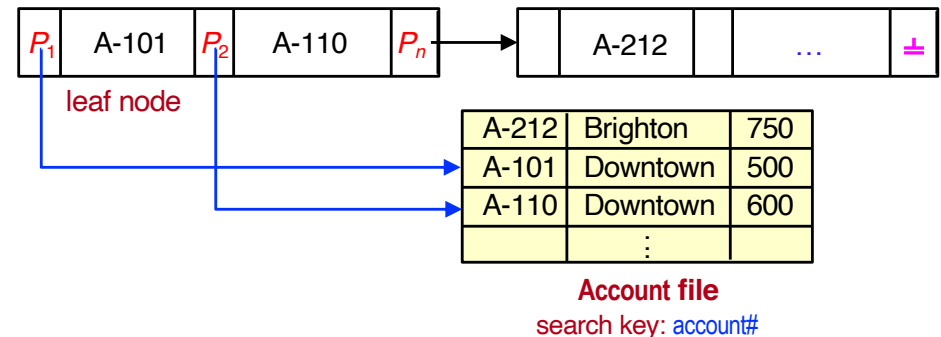
B⁺-TREE: NON-LEAF (INTERNAL) NODES

- Non-leaf (internal) nodes form a **multi-level index** on the leaf nodes.
- For a **non-leaf node** with **n pointers**:
 - First pointer P_1 :** All the search-key values in the subtree to which P_1 points are **less than K_1** .
 - Internal pointer:** For $2 \leq i \leq n-1$, all the search-key values in the subtree to which P_i points have values **greater than or equal to K_{i-1}** and **less than K_i** .
 - Last pointer P_n :** All the search-key values in the subtree to which P points are **greater than or equal to K_{n-1}** .



B⁺-TREE: LEAF (BOTTOM LEVEL) NODES

- For $i = 1, 2, \dots, n-1$, pointer P_i points to *either*
 - a file record with search-key value K_i , *or*
 - an indirection level (a list of pointers) each of which points to a file record with search-key value K_i .

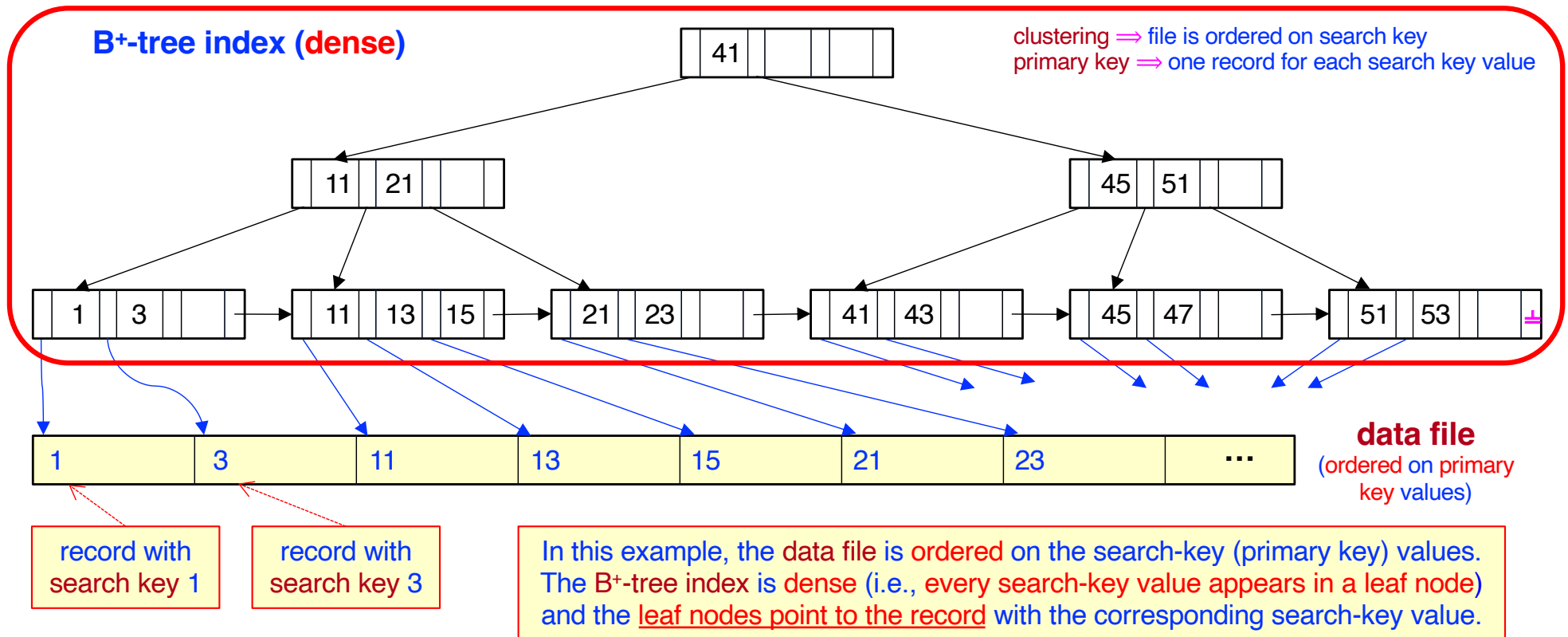


- Search-key values in a node are kept in sorted order.
 - If L_i and L_j are leaf nodes and $i < j$, L_i 's search-key values are less than L_j 's search-key values.
- The last pointer in a node, P_n , points to the next leaf node in the search-key order (right sibling node) or contains an end symbol $\perp$.

B⁺-TREE: OBSERVATIONS

- Since the inter-node connections are given by pointers, the “close” pages need not be “physically” close on secondary storage (i.e., there is no need to store pages sequentially).
- The non-leaf levels of the B⁺-tree form a sparse index (i.e., not every search-key value is present in the internal nodes).
- Since a B⁺-tree contains a relatively small number of levels (logarithmic in the size of the data file), search can be conducted efficiently.
- Insertions and deletions to the data file can be handled efficiently, as the index can be restructured in logarithmic time.

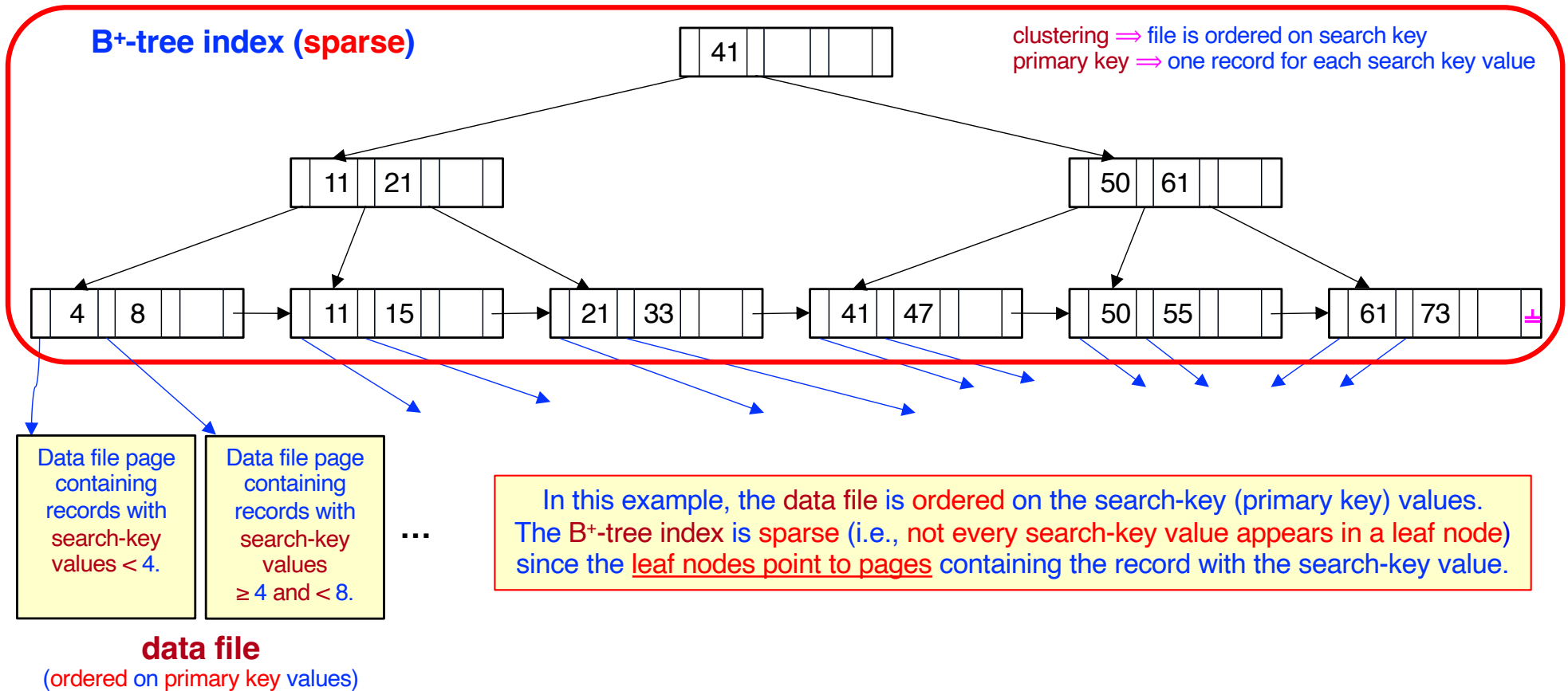
CLUSTERING, PRIMARY KEY B⁺-TREE (USING RECORD POINTERS)



Leaf nodes: contain all the primary key values (dense) and point to the record in the file with that value.

Data file: records are clustered (ordered) by the primary key.

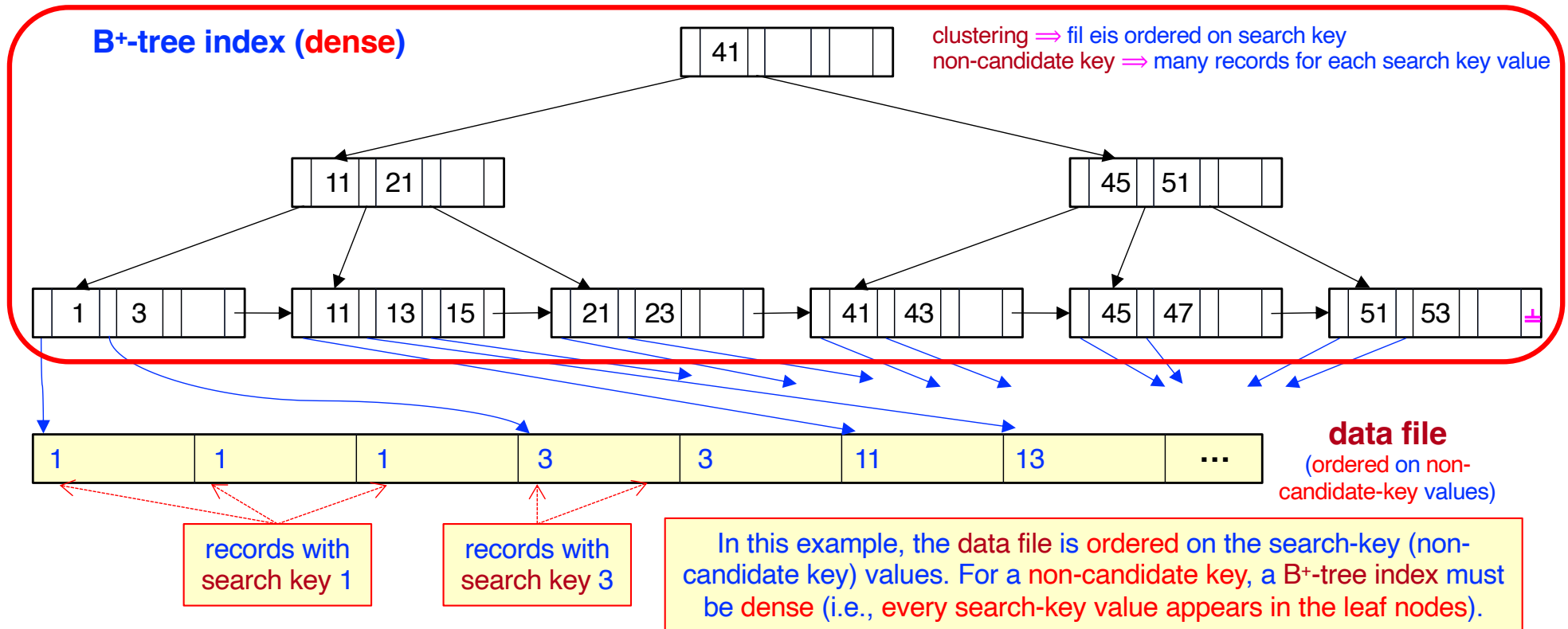
CLUSTERING, PRIMARY KEY B⁺-TREE (USING PAGE POINTERS)



Leaf nodes: contain only some of the primary key values (sparse) and point to a page with several records.

Data file: records are clustered (ordered) by the primary key.

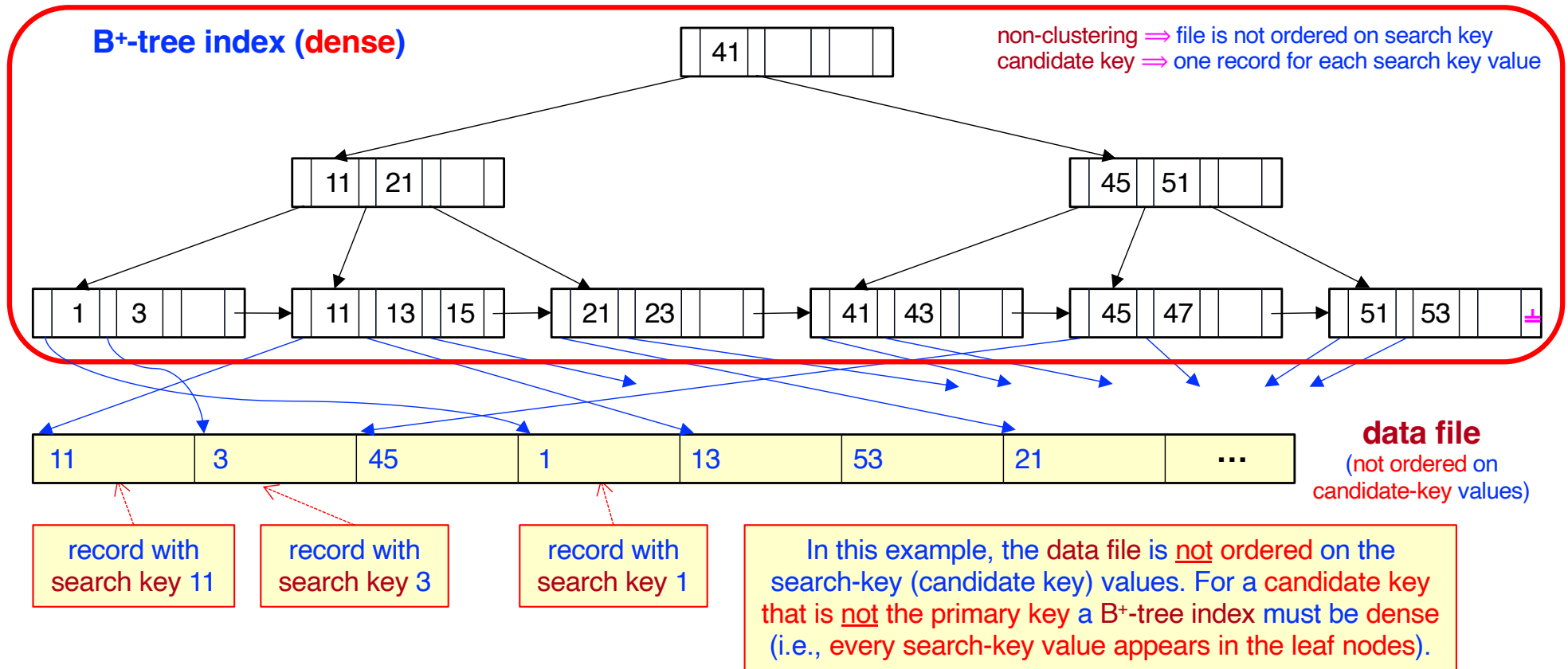
CLUSTERING, NON-CANDIDATE KEY B⁺-TREE (USING RECORD POINTERS)



Leaf nodes: contain all the unique non-candidate-key values (dense) and point to the first record in the file with that value.

Data file: records are clustered (ordered) by the non-candidate key, which may be duplicated in different records.

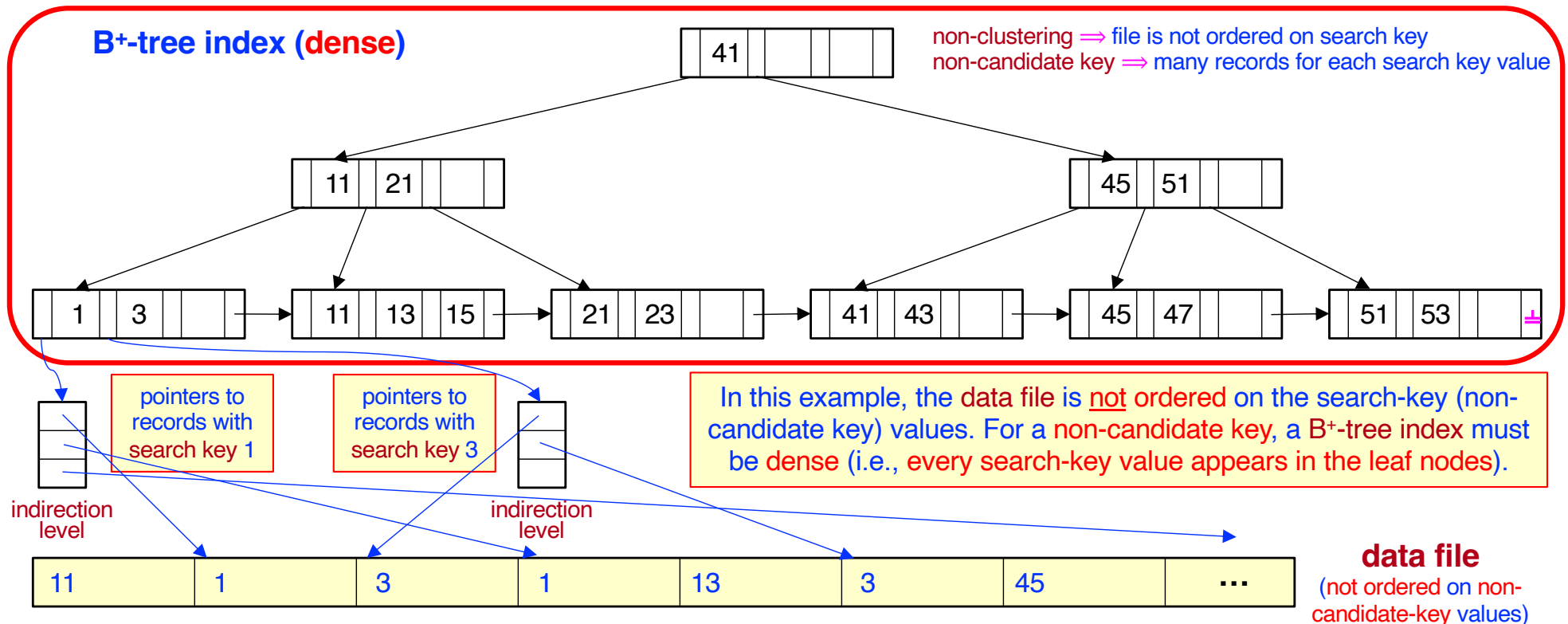
NON-CLUSTERING, CANDIDATE KEY B⁺-TREE (USING RECORD POINTERS)



Leaf nodes: contain all the candidate-key values (dense) and point to the record in the file with that value.

Data file: records are not clustered (not ordered) by the candidate key.

NON-CLUSTERING, NON-CANDIDATE KEY B⁺-TREE (USING INDIRECTION LEVEL)



Leaf nodes: contain all the unique non-candidate-key values (dense) which point to a list of pointers (indirection level) that point to the records in the file with that value.

Data file: records are not clustered (not ordered) by the non-candidate key.

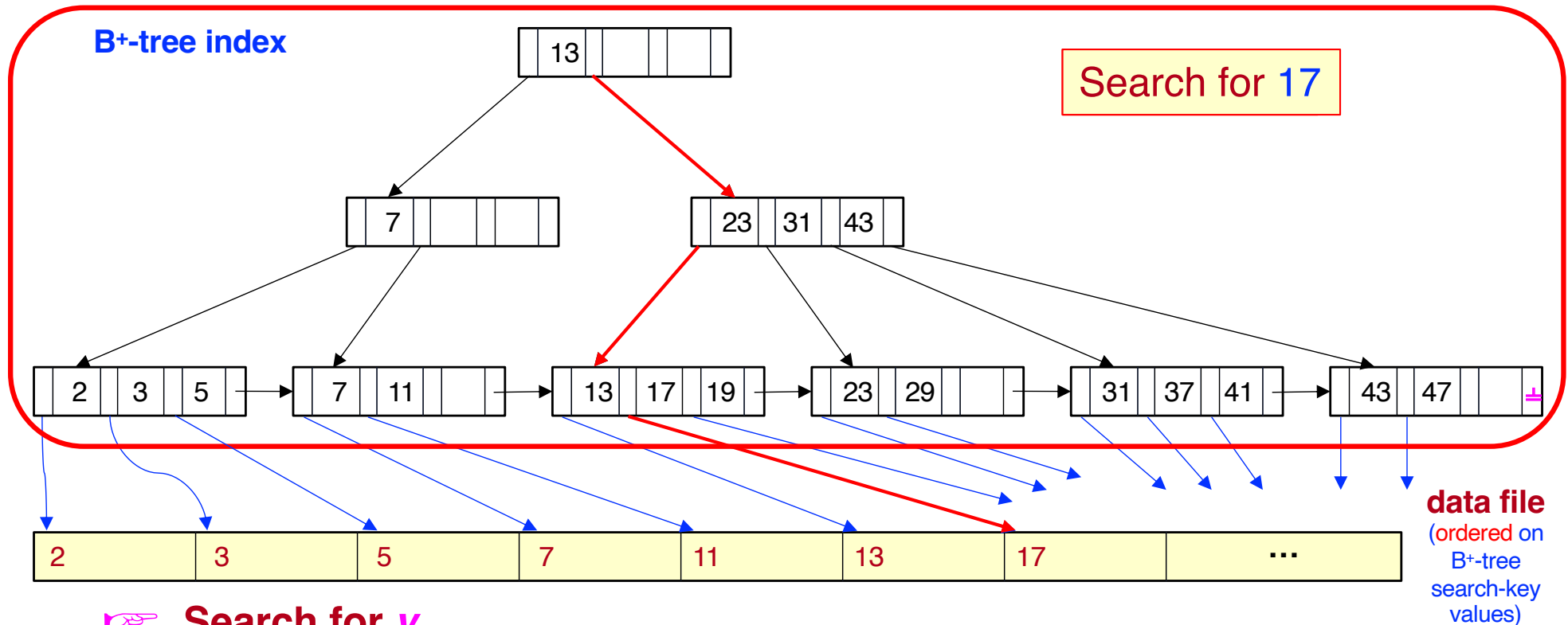
B⁺-TREE: QUERY

- Find all records with a search-key value of v .
 1. Start at the root node.
 - If $v < K_{n-1}$ (there are n pointers in the node, i.e., v is not larger than all values in the node) *follow pointer P_j* , where K_j is the smallest search-key value $> v$ (i.e., *follow the pointer to the left of K_j*).
 - Otherwise, if there is an entry with search-key value $K_j = v$, *follow pointer P_{j+1}* (i.e., *follow the pointer to the right of K_j*).
 - Otherwise, if $v \geq K_{n-1}$ the last value in the node, *follow pointer P_n* (i.e., *follow the last pointer in the node*).
 2. If the node reached by following the pointer in step 1 is **not a leaf node**, **repeat step 1** on the node reached, and follow the corresponding pointer.
 3. Reached a leaf node.
 - If for some i , key $K_i = v$ follow pointer P_i (i.e., *follow the pointer to the left of K_j*) to the desired record or list of pointers to records.
 - Else no record with search-key value v exists.

B⁺-TREE: QUERY (CONTD)

- In processing a query, a **path** is traversed in the tree from the **root to some leaf node**.
- If there are K search-key values in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
 - Assuming each node holds the minimum number of values.
- A node is generally the same size as a page, typically 4KB, and n is typically around 100 (assuming 40 bytes per index entry).
- With 1 million search-key values and $n = 100$, at most $\log_{50}(1,000,000) = 4$ nodes are accessed in a search.

B⁺-TREE QUERY: EXAMPLE



👉 Search for v

1. Start at root.
2. Find K_i such that $K_i > v$; *follow left pointer*; if no $K_i > v$, *follow last pointer* in node.
3. At each node, repeat step 2 until a leaf node is reached.
4. At leaf node:
 - If v is found, *follow left pointer* to the record or list of pointers to records.
 - Else, if v is not found, then no record with the search-key value v exists.

B⁺-TREE UPDATE

- When a record is inserted or deleted from a data file, indexes on the file must be updated.
- Updates to a record can be done as a deletion of the old record followed by an insertion of the updated record.

 **Only need to consider B⁺-tree insertion and deletion operations.**

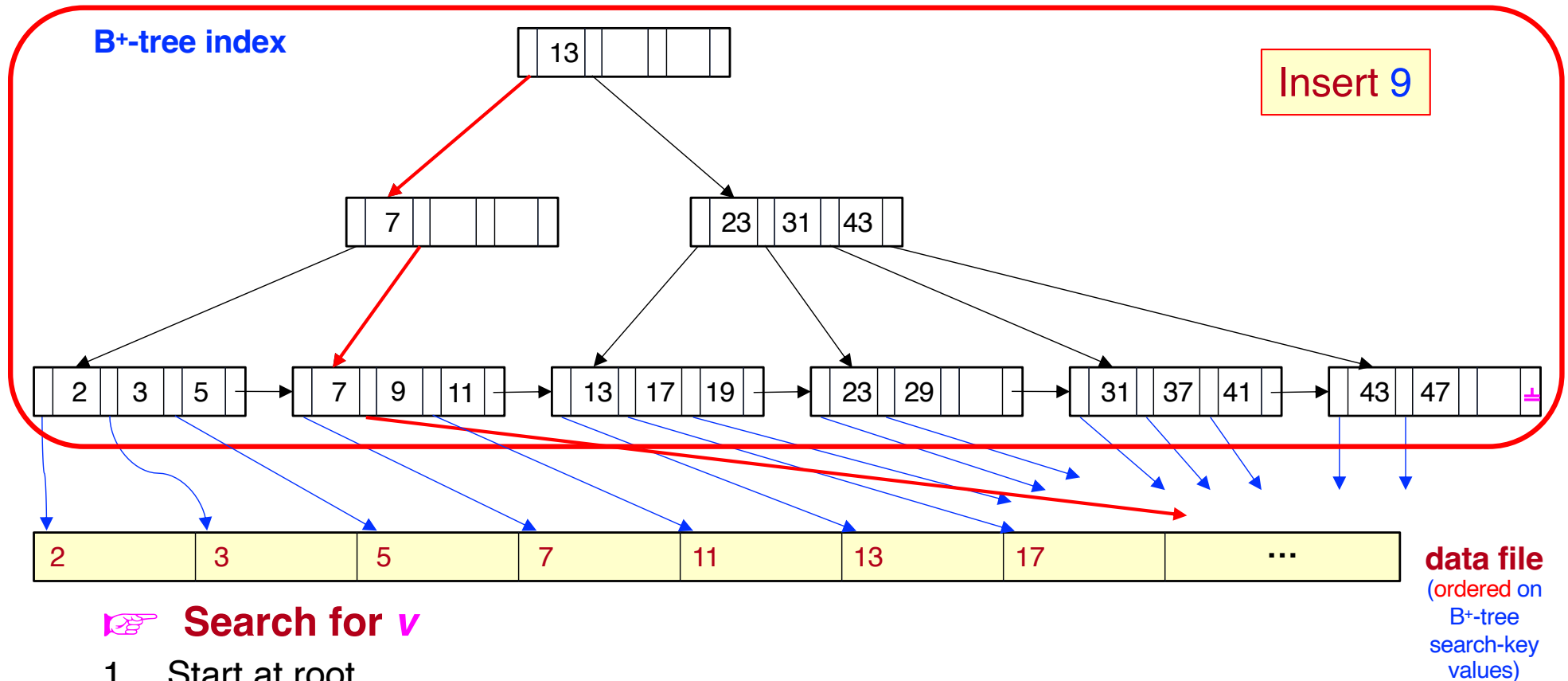
- To maintain the B⁺-tree properties, it may be necessary to
 - **split a node** when doing an insertion (**node overfull**).
 - **coalesce (combine) nodes** when doing a deletion (**node underfull**).

 **Must ensure that balance is preserved when doing an update (i.e., all paths from the root to the leaves are the same length).**

B⁺-TREE INSERTION

- Use query strategy to find the leaf node L where the value belongs.
- Put the value into L (a leaf node).
 - If L has enough space, *done!*
 - Else (*leaf node overfull*), must split L (into L and a new node leaf node L').
 - Place first $\lceil n/2 \rceil$ values into L , copy up value at $\lceil n/2 \rceil + 1$ and place values starting at $\lceil n/2 \rceil + 1$ up to and including the last value into L' .
 - Copy up: Insert an index entry (value at $\lceil n/2 \rceil + 1$, pointer to L') into the correct position in the (non-leaf) parent of L .
- Splits can happen recursively.
 - To split an index (*internal*) node N (into N and a new internal node N').
 - Place first $\lceil n/2 \rceil - 1$ values into N , push up value at $\lceil n/2 \rceil$ and place values starting at $\lceil n/2 \rceil + 1$ up to and including the last value into N' .
 - Push up: Insert an index entry (value at $\lceil n/2 \rceil$, pointer to N') into the correct position in the parent of N .
- Splits “grow” the tree; a root split increases the height.
 - B⁺-tree growth: The tree gets *wider* or *one level taller at the top*.

B⁺-TREE INSERTION: EXAMPLE



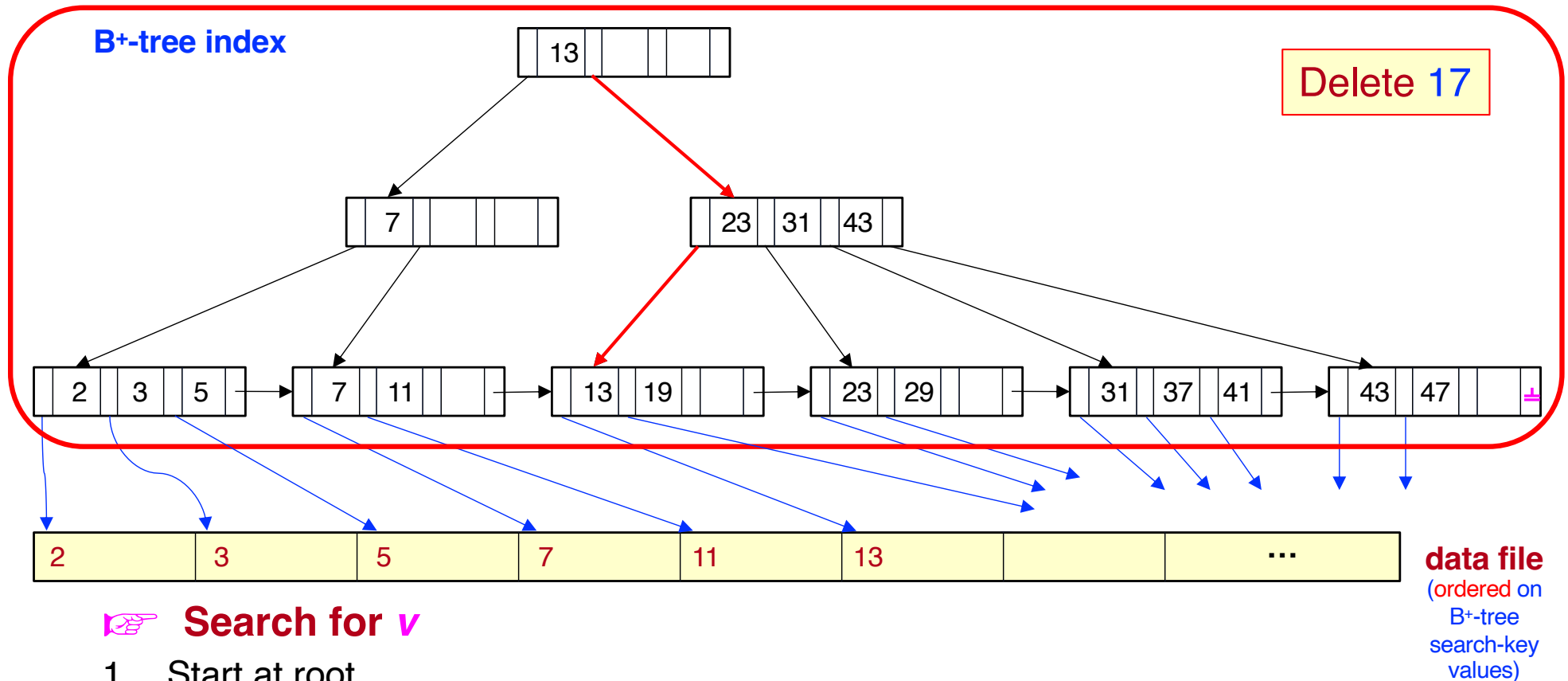
👉 Search for v

1. Start at root.
2. Find K_i such that $K_i > v$; *follow left pointer*; if no $K_i > v$, *follow last pointer* in node.
3. At each node, repeat step 2 until a leaf node is reached.
4. At leaf node:
 - If enough space, *insert in node in order*.
 - Else, (*leaf node overfull*), must *split*.

B⁺-TREE DELETION

- Use query strategy to find the leaf node L where the value is stored.
- Remove the leaf node entry.
 - If L has at least $\lceil (n-1)/2 \rceil$ values, *done!*
 - Else L has less than $\lceil (n-1)/2 \rceil$ values (*leaf node underfull*),
 - Try to re-distribute by borrowing values from a sibling node (an adjacent node to the right or the left *under the same parent*).
 - If re-distribution fails, merge L and a sibling.
- If a merge occurred, delete the entry (pointing to L or the sibling) from parent (non-leaf node) of L .
- A merge could propagate to the root, decreasing the height of the tree.
 - For non-leaf (internal) nodes, redistribution requires that a node have at least $\lceil n/2 \rceil$ pointers.
 - If this criterion cannot be met by re-distribution, then nodes must be merged.

B⁺-TREE DELETION: EXAMPLE

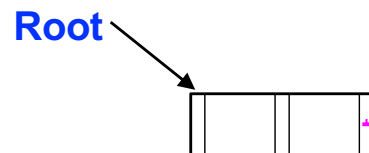


👉 Search for v

1. Start at root.
2. Find K_i such that $K_i > v$; *follow left pointer*; if no $K_i > v$, *follow last pointer* in node.
3. At each node, repeat step 2 until a leaf node is reached.
4. At leaf node:
 - If L has at least $\lceil (n-1)/2 \rceil$ values, *done!*.
 - Else, L has less than $\lceil (n-1)/2 \rceil$ values (*leaf node underfull*).

B⁺-TREE BULK LOADING

- Creating a B⁺-tree by repeatedly inserting records is very slow and can be costly (i.e., **need to read and write leaf nodes multiple times**).
- **Bulk Loading** can be done much more efficiently.
- **Initialization:** Sort all data entries (using external sorting); point to first (leaf) node in a new (root) page.



3	6	10	12	20	23	35
---	---	----	----	----	----	----

Sorted pages of data records; not yet in B⁺-tree

 **Only need to write each leaf node once; never have to read it.**

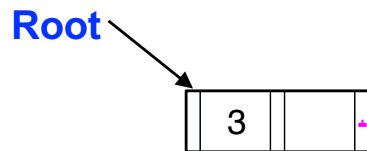
B⁺-TREE BULK LOADING: EXAMPLE

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 6 10 12 20 23 35

Load each leaf node with the minimum number of values (i.e., 1).

- Load first record into the root node.

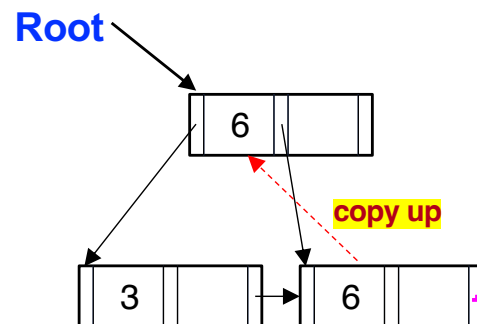


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 6 10 12 20 23 35

- Create a new leaf node and load the next record into it.
- Create a new root node.
- Copy up the search-key value from the new leaf node into the root node.

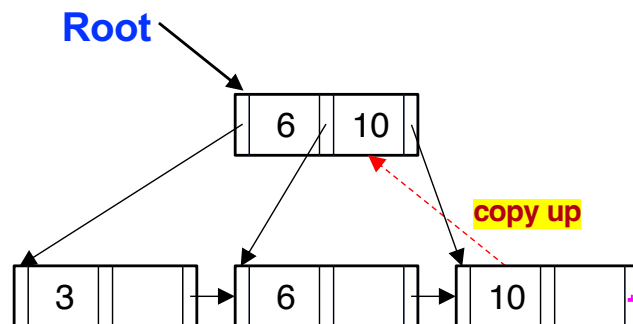


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 10 12 20 23 35

- Create a new leaf node and load the next record into it.
- Copy up the search-key value from the new leaf node into the parent node.

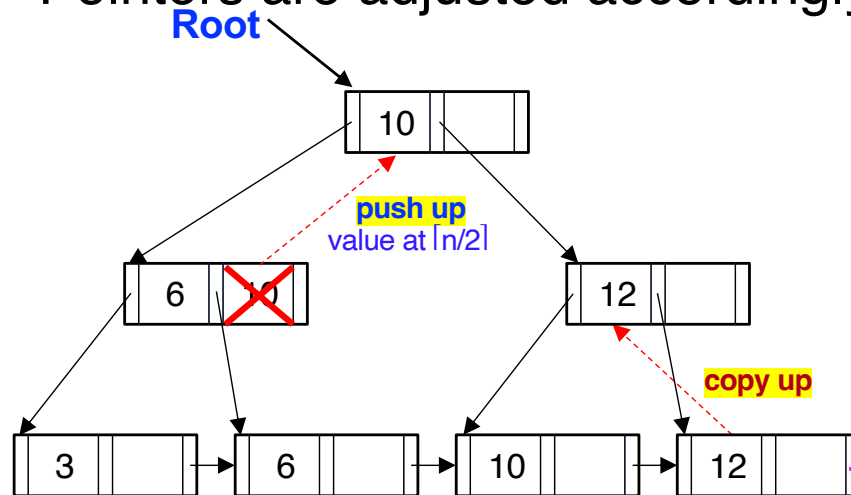


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 — 10 12 20 23 35

- Create a new leaf node and load the next record into it
- Copying up the search-key value from the new leaf node into the parent node causes the parent node to split and the tree to grow one level.
- Pointers are adjusted accordingly.

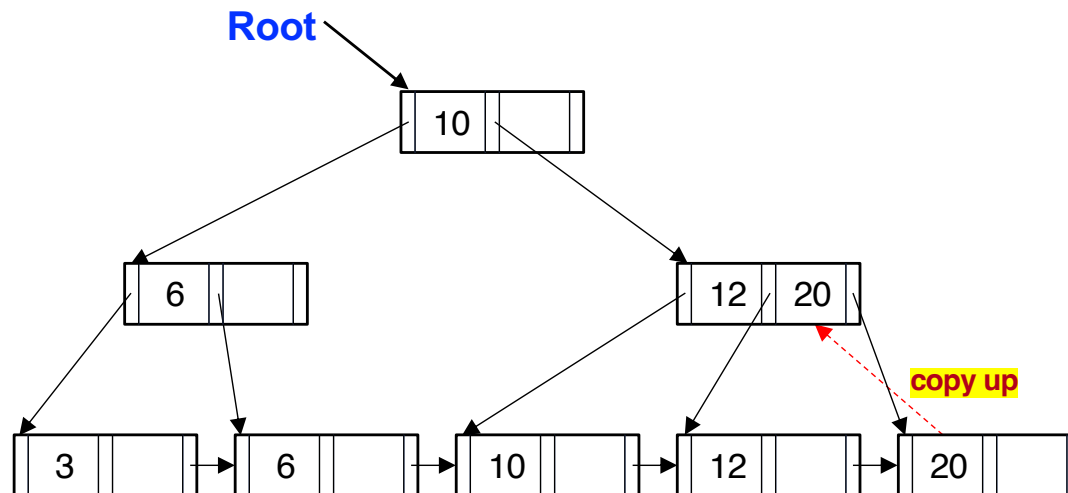


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 — 10 — 12 20 23 35

- Create a new leaf node and load the next record into it.
- Copy up the search-key value from the new leaf node into the parent node.

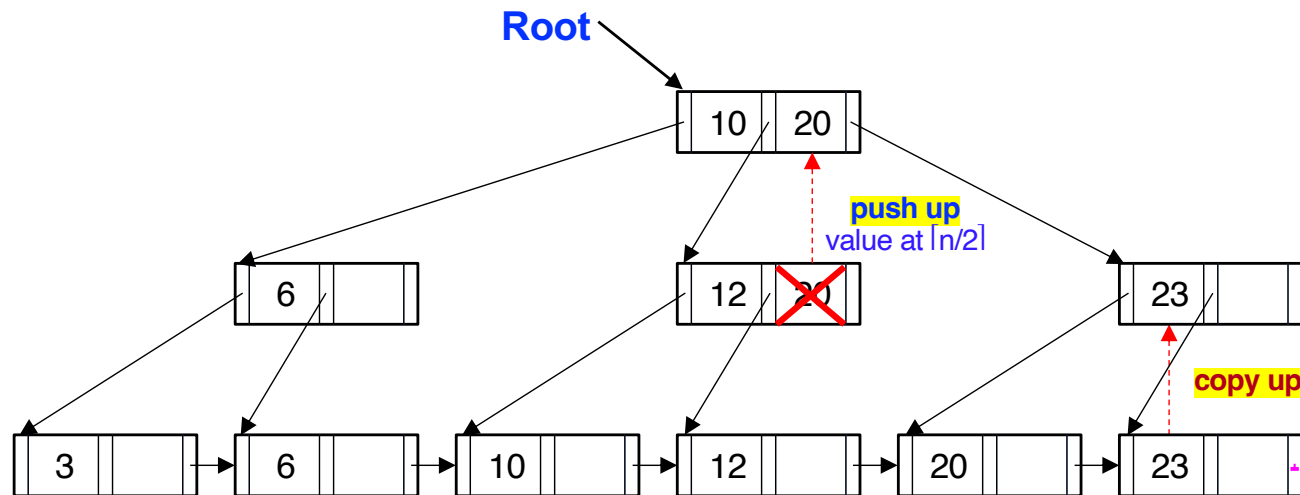


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 — 10 — 12 — 20 **23** 35

- Create a new leaf node and load the next record into it.
- Copying up the search-key value from the new leaf node into the parent node causes parent node to split.
- Pointers are adjusted accordingly.

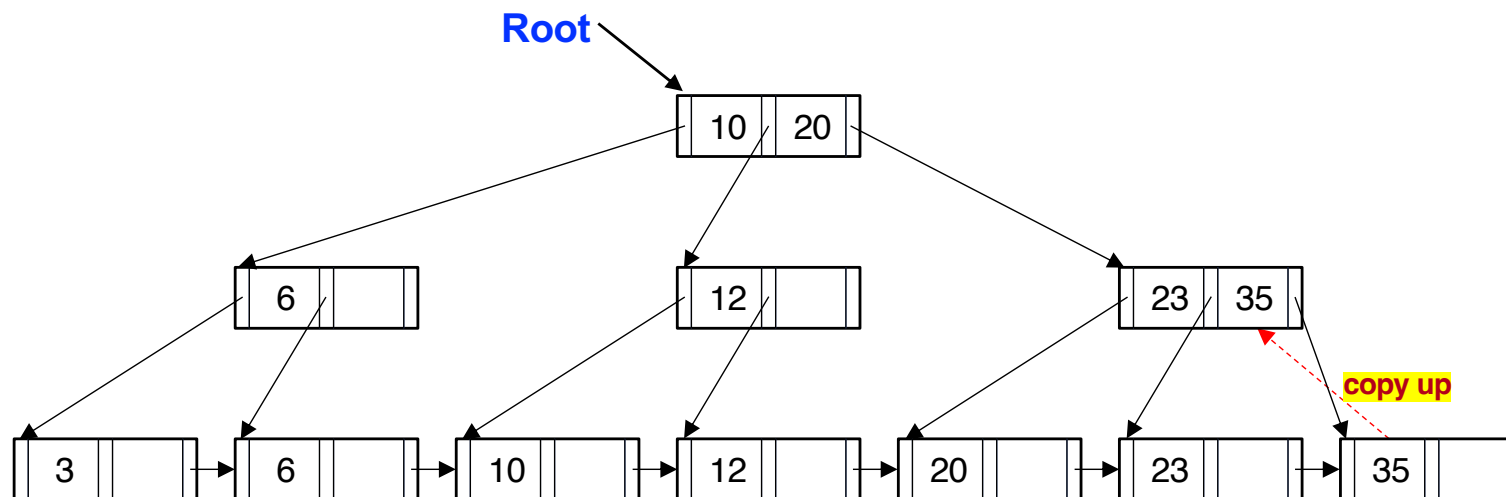


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 — 10 — 12 — 20 — 23 35

- Create a new leaf node and load the next record into it.
- Copy up the search-key value from the new leaf node into the parent node.

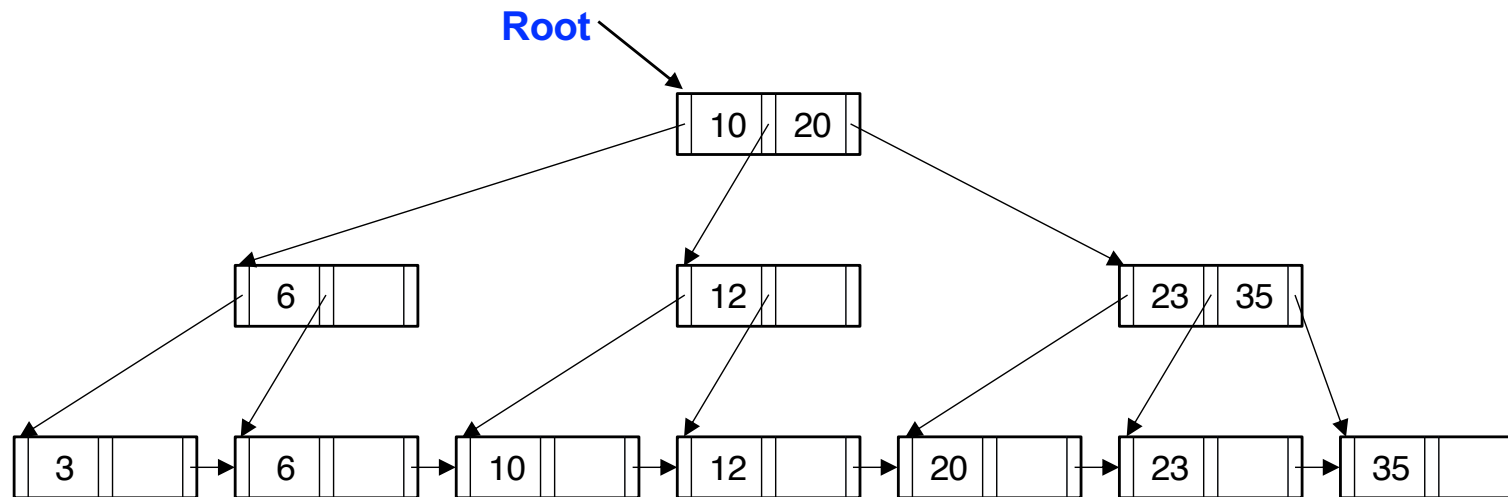


B⁺-TREE BULK LOADING: **EXAMPLE** (CONTD)

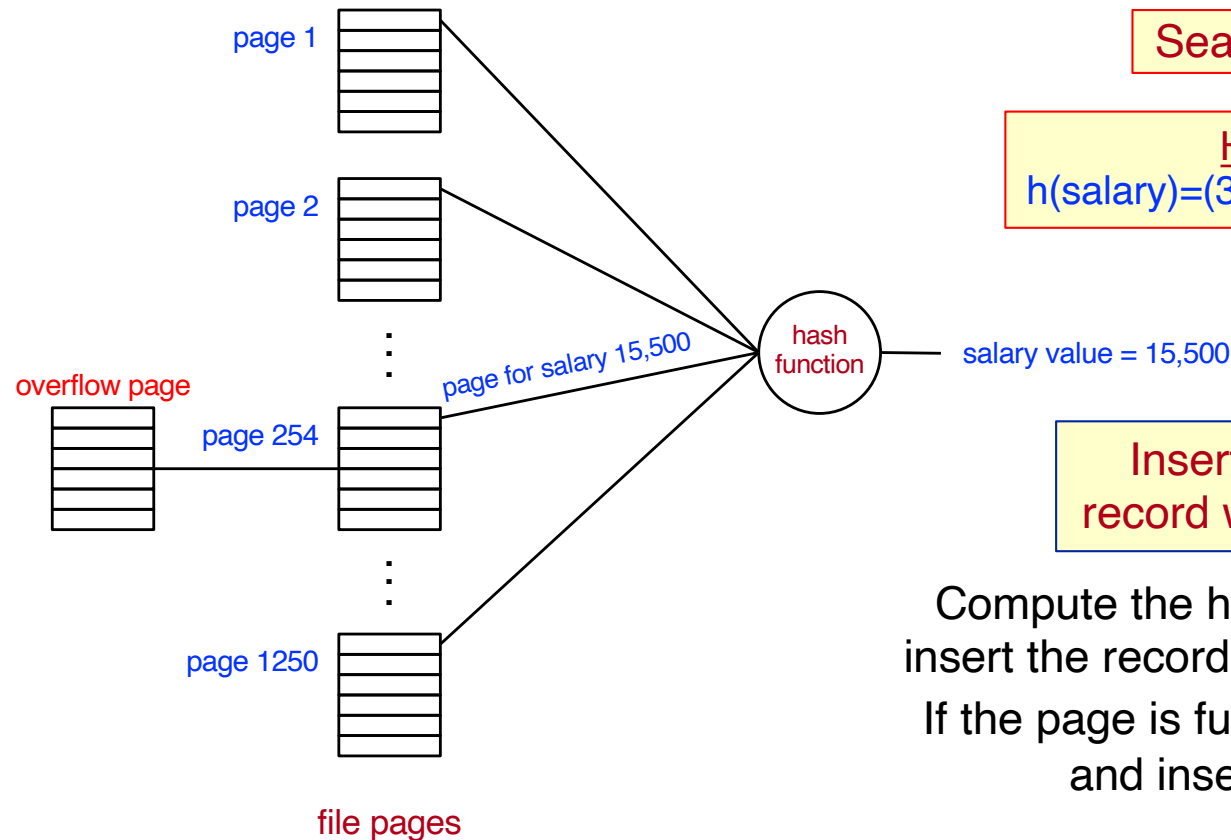
Assume $n = 3 \Rightarrow$ leaf/internal nodes between 1 and 2 values.

3 — 6 — 10 — 12 — 20 — 23 — 35

- Done.



RECALL: HASH FILE ORGANIZATION



👉 **Good for equality search on the hash value only!**

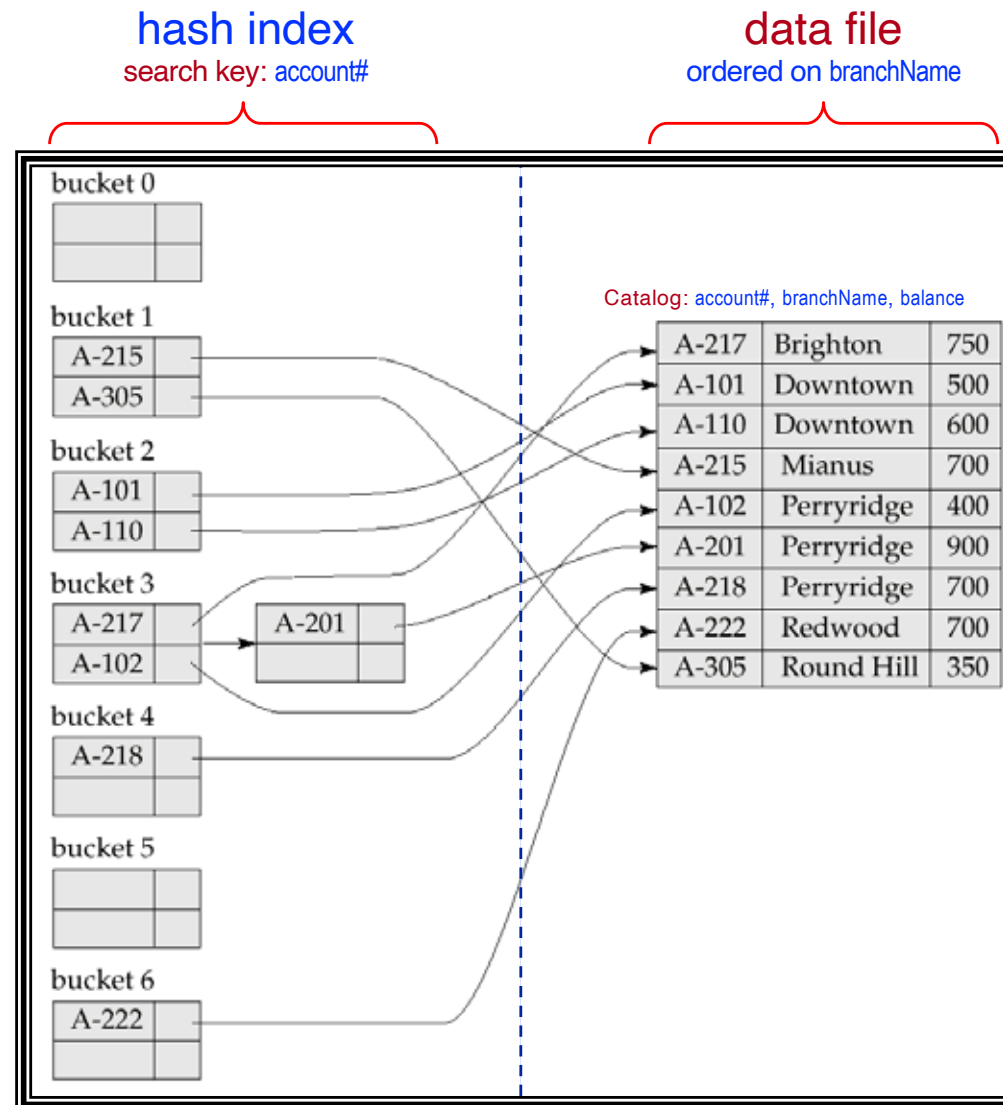
HASH INDEX

- A **hash index** organizes the search keys, with their associated record pointers, into a hash file organization.
 - ✎ The **index file**, not the data file, is the hash file.
 - ✎ Hash indexes are always secondary indexes.
- If the **data file** is organized using hashing on the primary key, a separate **primary key hash index** on it is not necessary.

Why?

- We want to build a hash index for an existing table using **static hashing**.
 - ✎ **Recall:** When using static hashing we **fix the number of pages** allocated to the hash file (index).

EXAMPLE HASH INDEX USING STATIC HASHING



STATIC HASHING DEFICIENCIES

- In **static hashing**, a function h maps search-key values to **a fixed set B of page addresses**.
 - Databases usually grow over time.
 - If the initial number of fixed pages is too small, performance will degrade due to too many overflows.
 - If the file size at some point in the future is anticipated to grow and the number of pages allocated accordingly, a significant amount of space will be wasted initially.
 - If the database shrinks, again space will be wasted.
 - One option is periodic re-organization of the file with a new hash function, but this is very expensive. **Why?**
- These problems can be avoided by using techniques that **allow dynamic modification of the hash function (i.e., dynamic modification of the number of pages to which we hash)**.

INDEX SELECTION CONSIDERATIONS

- **Types of accesses supported efficiently**
 - B⁺-tree indexes are used extensively in database systems for searching (~90% of uses cases).
 - Hash indexes are not used extensively for searching since they do not support range searches ⇒ used mainly for in-memory hashing.
 - Secondary indexes are not good for queries that retrieve many records as the same page may be retrieved multiple times.
- **Update time**
 - When a data file is modified, every index on the data file must be updated.
- **Space overhead**
 - The index should be, in general, much smaller than the data file.

INDEXING: SUMMARY

Indexing reduces the overhead of searching by trading space (storage overhead) for time (faster search).

Dense and sparse tree index

Dense: contain entries for every search-key value at the leaf level.

Sparse: contain entries for only some search-key values at the leaf level.

Clustering and non-clustering (secondary) index

Clustering: file is ordered on the search-key values.

Non-clustering: file is not ordered on the search-key values.

Indexing methods

B+-tree: use a search-key value to search a tree-index to find an index entry that points to the data file.

Hashing: apply a function to the search-key value to directly access an index entry that points to the data file.